L5-v2

2026年10月15日 15:12

核心变化：把 main.cpp 里的临时 for 循环正式抽象成 AgentLoop，并引入上下文分层组装。同时新增 3 个工具（Write / Edit / Bash）、观察者模式、消息合法性守卫和自检入口。

agent_loop.hpp	首次实现	L2 只声明，L5 补实现
agent_loop.cpp	新增文件	循环内核的完整实现
context_budget.hpp	首次部分实现	L2 声明，L5 只实现一半
context_budget.cpp	新增文件	上下文组装 + G8 守卫
main.cpp	大改	① 4 个工具；② 观察者接线；③ --selftest；④ 组合根

L4 的临时 for 循环

- 循环逻辑和渲染逻辑混在 main 里。
- std::cout 直接写在循环内部，分层被破坏。
- 没有观察者，UI 想接也接不进来。

```
// main.cpp (L4)
for (int round = 0; round < kMaxRounds; ++round) {
    ChatRequest req;
    req.system   = "...";
    req.messages = history;
    req.tools    = registry.defs();

    Renderer r;
    r.t0 = Clock::now();

    ChatResponse resp;
    try {
        resp = provider->stream(req, [&r](const ChatChunk& c) { r(c); return true; });
    } catch (const std::exception& e) {
        std::cerr << "\n[请求失败] " << e.what() << "\n";
        outcome = LoopOutcome::Error;
        break;
    }

    totalUsage.inputTokens  += resp.usage.inputTokens;
    totalUsage.outputTokens += resp.usage.outputTokens;
    history.push_back(resp.message);

    if (resp.finishReason == FinishReason::Length) { ... break; }

    if (resp.message.toolCalls.empty()) {
        finalText = resp.message.content;
        break;
    }

    for (const auto& tc : resp.message.toolCalls) {
        ++toolCallCount;
        ToolResult tr = registry.dispatch(tc, *ui);
        std::cout << ...;
        history.push_back(tr.toMessage());
    }
}
```

AgentLoop

关键变化：

维度	L4 main 里的 for	L5 AgentLoop::run
位置	藏在 main 里	独立的类，可复用
依赖	直接抓 provider / registry / ui	通过构造注入（引用），不持有
渲染	直接 std::cout	通过 LoopHooks 抛出事件
上下文	手写 req.system / req.messages	由 ContextAssembler::assemble 统一
状态	局部变量 history	Turn&（外部持有）+ 成员 spentTokens_
错误回喂	没有（直接 break）	把错误作为 observation 塞回历史，continue

```cpp
// agent_loop.cpp
LoopOutcome AgentLoop::run(Turn& turn, const LoopHooks& hooks) {
    for (int step = 0; step < limits_.maxSteps; ++step) {

        // ---- A. 组装上下文 (环内每一步都重新组装) ----
        ChatRequest req = assembler_.assemble(turn);
        req.maxTokens = std::min(req.maxTokens, assembler_.config().maxOutputTokens);

        // ---- B. 调模型 ----
        ChatResponse resp;
        try {
            resp = provider_.stream(req, [&](const ChatChunk& c) -> bool {
                if (hooks.onChunk) hooks.onChunk(c);   // ★ 观察者接入点
                return true;
            });
        } catch (const std::exception& e) {
            if (hooks.onError) hooks.onError(e.what());
            if (hooks.onNotice) hooks.onNotice("模型请求失败，已回喂给模型，继续下一步");
            Message m;
            m.role    = Role::User;
            m.content = std::string("[系统] 模型请求失败: ") + e.what() +
                        "。请继续，或换一种方式。";
            turn.history.push_back(std::move(m));
            continue;
        }

        // ---- C. 记账 + 存历史 ----
        turn.history.push_back(resp.message);
        spentTokens_ += resp.usage.total();
        total_.inputTokens += resp.usage.inputTokens;
        // ...

        // ---- D. 结束判定 ----
        if (!resp.message.hasToolCalls()) {
            turn.finalText = resp.message.content;
            return LoopOutcome::Completed;
        }

        // ---- E. 执行工具并回喂 ----
        static NoUI kNoUI;
        IUIPort& ui = assembler_.uiPort() ? *assembler_.uiPort()
                                          : static_cast<IUIPort&>(kNoUI);

        int done = 0;
        for (const auto& tc : resp.message.toolCalls) {
            if (hooks.onToolStart) hooks.onToolStart(tc);
            ToolResult r = registry_.dispatch(tc, ui);
            if (hooks.onToolDone) hooks.onToolDone(r);
            turn.history.push_back(r.toMessage());
            ++done;
        }

        if (hooks.onNotice) { ... }
    }
    return LoopOutcome::StepLimitReached;
}
```

```cpp
struct Turn {
    std::string input;                          // 用户输入（永远最后进上下文）
    std::string systemPrompt;                   // L0
    std::vector<Message> history;               // L5
    std::vector<Message> activeFiles;           // L4（编辑后必须立即刷新）
    std::vector<Message> retrieved;             // L3

    Usage       lastUsage;
    int         spentTokens = 0;
    std::string finalText;                      // 任务完成时的最终回答
};
```

```cpp
enum class Layer {
    L0_System,        // 恒定          → 缓存前缀，永不裁剪
    L1_Project,       // CLAUDE.md
    L2_RepoMap,       // 符号骨架
    L3_Retrieved,     // 检索命中
    L4_ActiveFiles,   // 正在编辑的文件全文
    L5_History,       // 对话历史（可能已被压缩）
    L6_UserInput,     // 永远最后、永远完整
};
```

```cpp
struct LoopHooks {
    std::function<void(const ChatChunk&)>    onChunk;      // 流式增量
    std::function<void(const ToolCall&)>     onToolStart;
    std::function<void(const ToolResult&)>   onToolDone;
    std::function<void(const std::string&)>  onNotice;
    std::function<bool(const std::string&)>  confirm;      // 权限询问
    std::function<void(const std::string&)>  onError;
};
```

```cpp
Renderer renderer;
int toolCalls = 0;
int chunks = 0;              // onChunk 被调用的次数（观察者确实在工作）

LoopHooks hooks;
hooks.onChunk = [&](const ChatChunk& c) { ++chunks; renderer(c); };
hooks.onToolStart = [](const ToolCall& tc) {
    std::cout << "[事件] onToolStart: " << tc.name << "\n" << std::flush;
};
hooks.onToolDone = [&](const ToolResult& r) {
    std::cout << "[事件] onToolDone: " << (r.isError ? "错误" : "成功") << "\n"
              << (r.isError ? "[工具失败] " : "[工具返回] ")
              << r.content.substr(0, 400)
              << (r.content.size() > 400 ? " ..." : "") << "\n\n" << std::flush;
    ++toolCalls;
};
hooks.onNotice = [](const std::string& msg) {
    std::cout << "[事件] onNotice: " << msg << "\n" << std::flush;
};
hooks.onError = [](const std::string& e) {
    std::cerr << "\n[请求失败] " << e << "\n" << std::flush;
};
// hooks.confirm 本讲未接：环还没有"需要用户二次确认"的决策点（L6 的闸门）
```

关键变化：

上下文预算：

contextWindow配额比例：

```
contextWindow (200K)
  ├─ reserveForReply (16K)   ← 给模型留的回复空间，不能占
  ├─ maxOutputTokens (8K)    ← 单次 max_tokens 上限
  └─ available()             ← 真正可用于"输入"的额度
       ├─ L0_System   (5%)   ← 系统提示
       ├─ L1_Project  (8%)   ← 项目知识
       ├─ L2_RepoMap  (5%)   ← 仓库地图
       ├─ L3_Retrieved(25%)  ← 检索命中
       ├─ L4_Active   (30%)  ← 活动文件
       └─ L5_History  (剩余) ← 历史（兜底拿剩下的）
```

顺序即缓存前缀：

上下文的排列顺序不可随意，它同时服务于两个目标：

目标	要求	结果
省钱省延迟	越稳定的内容越靠前	L0/L1/L2 严格固定在前
模型注意力	越相关的内容越靠后	L6 用户输入永远最后

超预算降级链：

```
Full (全文)
  ↓ 超预算
Signatures (函数签名)
  ↓ 还超
SymbolNames (符号名 + 路径)
  ↓ 还超
PathOnly (仅路径)
  ↓ 还超
Drop (丢弃)
```

```
enum class Layer {
    L0_System,       // 恒定          → 缓存前缀，永不裁剪
    L1_Project,      // CLAUDE.md
    L2_RepoMap,      // 符号骨架
    L3_Retrieved,    // 检索命中
    L4_ActiveFiles,  // 正在编辑的文件全文
    L5_History,      // 对话历史（可能已被压缩）
    L6_UserInput,    // 永远最后、永远完整
};
```

层	内容	稳定性	可裁剪性
L0	系统提示	会话内恒定	**永不裁剪**
L1	项目知识（CLAUDE.md）	会话内恒定	可裁剪
L2	仓库地图（符号骨架）	会话内恒定	可裁剪
L3	检索命中	每轮可变	可降级
L4	活动文件全文	编辑后刷新	可降级
L5	对话历史	每轮追加	可压缩
L6	用户输入	每轮变	**永不裁剪**

BudgetConfig —— 预算配置

```
struct BudgetConfig {
    int contextWindow   = 200'000;   // 模型总窗口
    int reserveForReply = 16'384;    // 给回复留的位置
    int maxOutputTokens = 8'192;     // 单次 max_tokens 上限

    double sysFrac = 0.05, projectFrac = 0.08, repoMapFrac = 0.05,
           retrievedFrac = 0.25, activeFrac = 0.30;
    // 剩余部分全部归历史层 (L5)

    int available() const noexcept {
        return contextWindow - reserveForReply - maxOutputTokens;
    }

    int forLayer(Layer l) const noexcept {
        const int a = available();
        switch (l) {
            case Layer::L0_System:      return int(double(a) * sysFrac);
            case Layer::L1_Project:     return int(double(a) * projectFrac);
            case Layer::L2_RepoMap:     return int(double(a) * repoMapFrac);
            case Layer::L3_Retrieved:   return int(double(a) * retrievedFrac);
            case Layer::L4_ActiveFiles: return int(double(a) * activeFrac);
            default:                    return a;      // L5 拿剩下的
        }
    }
};
```

Degrade —— 降级链

```
enum class Degrade { Full = 0, Signatures = 1, SymbolNames = 2, PathOnly = 3, Drop = 4 };
```

级别	保留内容	示例
Full	全文	完整函数体
Signatures	函数/类签名	int add(int a, int b);
SymbolNames	符号名 + 路径	add @ foo.cpp
PathOnly	仅路径	foo.cpp
Drop	丢弃	（空）

```
struct LayerUsage {
    int l0 = 0, l1 = 0, l2 = 0, l3 = 0, l4 = 0, l5 = 0, l6 = 0;
    int total = 0;
};
```

```cpp
ContextAssembler::LayerUsage ContextAssembler::measure(const Turn& turn) const {
    LayerUsage u;
    u.l0 = estimateTokens(system_);
    u.l1 = estimateTokens(project_);
    u.l2 = estimateTokens(repoMap_);
    for (const auto& c : turn.retrieved)   u.l3 += estimateTokens(c.content);
    for (const auto& f : turn.activeFiles) u.l4 += estimateTokens(f.content);
    for (const auto& m : turn.history)     u.l5 += estimateTokens(m.content);
    u.l6 = estimateTokens(turn.input);
    u.total = u.l0 + u.l1 + u.l2 + u.l3 + u.l4 + u.l5 + u.l6;
    return u;
}
```

```cpp
constexpr int kCharsPerToken = 4;
int estimateTokens(std::string_view s) {
    return int(s.size() / kCharsPerToken) + 1;
}
```

组装算法：

```cpp
ChatRequest ContextAssembler::assemble(const Turn& turn) {
    ChatRequest req;

    // ---- ① L0: 系统提示（缓存前缀，必须字节级稳定）----
    req.system = system_;
    if (!project_.empty()) req.system = *req.system + "\n\n# 项目知识\n" + project_;
    if (!repoMap_.empty()) req.system = *req.system + "\n\n# 仓库地图\n" + repoMap_;

    // ---- ② L5: 历史（★ 过一遍合法性守卫）----
    for (auto& m : guardMessageLegality(turn.history)) req.messages.push_back(std::move(m));

    // ---- ③ L6: 用户输入永远最后、永远完整 ----
    req.messages.push_back(Message{});
    req.messages.back().role    = Role::User;
    req.messages.back().content = turn.input;

    // ---- ④ 工具表：属于"模型能看到什么"的一部分 ----
    req.tools = tools_;

    req.cacheControl = CacheControl{true, 1024};
    return req;
}
```

消息合法性守卫：

协议硬要求：每一条 `tool` 消息必须紧跟"请求了它那个 `id` 的 `assistant` 消息"，而且这个 `assistant` 消息不能被别的消息隔开。

三类破坏方式：

类型	示例	后果
① 孤儿 tool	裁剪时删了 assistant，留下 tool	400
② 配对窗口被打断	中间插了一条 user（如系统错误回喂）	400
③ 只回一半	一次请求 3 个工具，只回填 1 条结果	400

```cpp
std::vector<Message> guardMessageLegality(const std::vector<Message>& msgs) {
    std::vector<Message> out;
    out.reserve(msgs.size());

    std::set<std::string> waiting;   // 待配对的 tool_call_id 集合

    for (const auto& m : msgs) {
        if (m.role == Role::Assistant && !m.toolCalls.empty()) {
            // 新的一批工具调用开始：登记整批 id，重置配对窗口
            waiting.clear();
            for (const auto& tc : m.toolCalls) waiting.insert(tc.id);
            out.push_back(m);

        } else if (m.isToolResult()) {
            // ★ 只有"引用的 id 还在待配对集合里"才算合法
            if (waiting.find(m.toolCallId) == waiting.end()) {
                continue;                       // 孤儿 / 重复 / 伪造 → 丢弃
            }
            out.push_back(m);
            waiting.erase(m.toolCallId);        // 消掉一个：并行调用要消 N 次

        } else {
            // 任何其他消息都会**关闭配对窗口**。
            waiting.clear();
            out.push_back(m);
        }
    }
    return out;
}
```

预算完整数据流：

```
BudgetConfig (配置)
  |
  ├─ contextWindow = 200K
  ├─ reserveForReply = 16K
  ├─ maxOutputTokens = 8K
  └─ 各层比例: sysFrac / projectFrac / repoMapFrac / retrievedFrac / activeFrac

available() = 200K - 16K - 8K = 176K
  |
  ├─ forLayer(L0_System)      = 176K * 5%  = 8,800
  ├─ forLayer(L1_Project)     = 176K * 8%  = 14,080
  ├─ forLayer(L2_RepoMap)     = 176K * 5%  = 8,800
  ├─ forLayer(L3_Retrieved)   = 176K * 25% = 44,000
  ├─ forLayer(L4_ActiveFiles) = 176K * 30% = 52,800
  └─ forLayer(L5_History)     = 176K * 27% = 47,520

assemble(turn)
  |
  ├─ L0 system_          → 直接放
  ├─ L1 project_         → fitTo(预算 14,080)
  ├─ L2 repoMap_         → fitTo(预算 8,800)
  ├─ L3 turn.retrieved   → 每条 fitTo(预算 44,000 / N)
  ├─ L4 turn.activeFiles→ 每条 fitTo(预算 52,800 / N)
  ├─ L5 turn.history     → 按预算裁剪 (L7 触发压缩)
  ├─ L6 turn.input       → 直接放
  └─ tools               → 直接放
  |
  ▼
guardMessageLegality(messages)  ← 清理孤儿 tool 消息
  |
  ▼
ChatRequest
  |
  ▼
measure(turn) → LayerUsage
  |
  ▼
/cost 可视化 (L12)
```

V2完整数据流

```
用户输入 "把 work_target.txt 里的 add 改名为 sum，然后跑测试确认"
  │
  ▼ main.cpp 组合根
装配 ToolRegistry (4 个工具) + ContextAssembler + AgentLoop
接线 LoopHooks (6 个观察者回调)
  │
  ▼ loop.run(turn, hooks)
for (step = 0; step < maxSteps; ++step) {
    │
    ├─ A. assembler.assemble(turn)
    │       ├─ L0 system
    │       ├─ L5 history (★ 过 guardMessageLegality)
    │       ├─ L6 user_input
    │       └─ tools
    │
    ├─ B. provider.stream(req, [&](c) { hooks.onChunk(c); return true; })
    │       └─ 模型返回 tool_calls: [Read, Edit, Bash]
    │
    ├─ C. turn.history.push_back(resp.message)
    │     记账: spentTokens_ += resp.usage.total()
    │
    ├─ D. resp.message.hasToolCalls() → true → 继续
    │
    └─ E. for (tc : resp.message.toolCalls) {
              hooks.onToolStart(tc)
              ToolResult r = registry.dispatch(tc, ui)
              hooks.onToolDone(r)
              turn.history.push_back(r.toMessage())   ← ★ 成对回填
          }
}
  │
  ▼ 下一轮 step
assembler.assemble(turn) 重新组装 (含上一轮的工具结果)
  │
  ▼ 模型看到工具结果，给出最终回答
resp.message.hasToolCalls() → false
  │
  ▼ turn.finalText = resp.message.content
return LoopOutcome::Completed
  │
  ▼ main.cpp 收尾
assembler.measure(turn) → L0/L5/L6 占用
loop.spentTokens() → token 统计
```

四个工具

工具	唯一要点	设计意图
Read	`persistThresholdChars = SIZE_MAX` 豁免落盘	避免"读文件 → 落盘 → 再读落盘文件"的循环依赖
Write	描述里写明"先 Read 再写"	否则模型是盲写——它会凭记忆重写文件，悄悄丢掉没读到的行
Edit	`old_str` 必须在文件中唯一出现	不唯一就失败——用"失败"换"准确性"，模型被迫给出更多上下文
Bash	退出码必须进结果	把"非 0"当成功返回空输出，会让模型得到"文件里没有"这个错误结论

>